

DataLab 实验报告

姓名：_____ 学号：_____

测试结果

在仓库根目录运行 `python3 test.py -v`，编译无警告，所有函数的正确性和运算符检查均通过，总分 37/37。

函数	得分	函数	得分
bitAnd	1/1	bitXor	1/1
sameSign	2/2	logTwo	4/4
byteSwap	4/4	reverse	3/3
logicalShift	3/3	leftBitCount	4/4
float_i2f	4/4	floatScale2	4/4
float64_f2i	3/3	floatPower2	4/4

```
rm -f *.o btest fshow ishow *~ yaccTab.py lextab.py result.txt .autograder_result
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
gcc -O0 -Wall -std=gnu99 -o fshow fshow.c
gcc -O0 -Wall -std=gnu99 -o ishow ishow.c
Make success.
bitAnd      1/1:    PASS    detail: Pass using 4 operations including {'|', '~'} and {'int'}.
bitXor      1/1:    PASS    detail: Pass using 7 operations including {'&', '~'} and {'int'}.
sameSign    2/2:    PASS    detail: Pass using 6 operations including {'!', '^', '>'} and {'if',
logTwo      4/4:    PASS    detail: Pass using 18 operations including {'|', '<<', '>', '>>'} and
byteSwap    4/4:    PASS    detail: Pass using 11 operations including {'<<', '^', '&', '>>'} and
reverse     3/3:    PASS    detail: Pass using 5 operations including {'<<', '&', '~', '>>'} and
logicalShift 3/3:    PASS    detail: Pass using 6 operations including {'<<', '&', '~', '>>'} and
leftBitCount 4/4:    PASS    detail: Pass using 32 operations including {'!', '+', '&', '>>', '<<'
float_i2f    4/4:    PASS    detail: Pass using 29 operations including {'!', '|', '+', '&', '>>',
floatScale2  4/4:    PASS    detail: Pass using 15 operations including {'!', '|', '+', '&', '>>',
float64_f2i  3/3:    PASS    detail: Pass using 19 operations including {'!', '&', '>>', '<<', '>'
floatPower2  4/4:    PASS    detail: Pass using 9 operations including {'+', '<<', '>', '-', '<'}
Total points: 37
```

解题报告

主要思路

我觉得 `byteSwap`、`reverse` 和 `float_i2f` 最能体现这次实验的内容：前两题分别用掩码和逐位移动处理位串；后一题需要把整数拆成浮点数的符号、阶码和尾数，还要处理舍入。下面按整型题、浮点题说明。代码只修改了 `bits.c`，没有使用额外头文件或函数调用。

整型与位运算题

- `bitAnd`：用德摩根律，把 `x & y` 改写成 `~(~x | ~y)`，满足只能使用 `~`、`|` 的限制。

- bitXor: 异或只在两位不同时为 1。用 $\sim(\sim x \ \& \ \sim y)$ 排除两位都为 0，再用 $\sim(x \ \& \ y)$ 排除两位都为 1，最后把两部分相与。
- sameSign: 先单独处理零，因为零既不算正数也不算负数；两个非零数异或后，检查最高符号位是否不同。
- logTwo: 答案是最高的 1 所在的位置。由于题目不允许循环，依次判断高 16、8、4、2、1 位是否包含最高的 1，并累计右移的位数。
- byteSwap: $n \ll 3$ 把字节编号换成位移量；右移后与 0xFF 取出目标字节。两个字节的异或结果 `delta` 只记录它们不同的位。把 `delta` 移回两个字节的位并分别与原数异或，便完成交换，其余字节不变。
- reverse: 循环 32 次，每次取出输入的最低位，把已有结果左移一位，再把该位接到结果右端；输入同时右移一位。
- logicalShift: 对负数直接使用 $\gg$ 会在左边补 1。先右移，再用掩码清除左侧补入的位，得到逻辑右移的结果。
- leftBitCount: 把输入取反后，原来连续的前导 1 就成为前导 0。分段判断前 16、8、4、2、1 位是否全为 0；确认一段后累计长度并左移，继续检查下一段。

浮点编码题

- float_i2f: 先求符号和整数绝对值，再找到最高的 1，确定规格化指数。有效位超过单精度的 24 位时，保留高位并检查舍弃的低位：大于一半进位，恰好一半时让保留结果的末位为偶数。舍入产生的进位也可以进入阶码。
- floatScale2: 阶码全 1 时直接保留 NaN 或无穷；阶码全 0 时左移小数部分；普通规格化数则将阶码加 1，若增加后达到全 1 就返回相应符号的无穷。
- float64_f2i: 从高低两个 32 位参数中取出双精度的符号、阶码和尾数。扣除 1023 的阶码偏置后，按实际指数移动尾数，只保留整数部分，实现向零截断；无穷、NaN 和超过整型范围的数返回题目指定的标记。
- floatPower2: 按指数分区直接拼位串：小于 -149 返回 0；-149 到 -127 设置非规格化尾数；-126 到 127 写入偏置后的阶码；更大时返回正无穷。

收获

这次实验让我把“一个数的数值”和“存储它的位串”区分开了。整型题重点是通过掩码选出要处理的位置；浮点题则要先分清规格化数、非规格化数和特殊值，再决定移动或舍入哪些位。

参考资料

- 仓库 [README.md](#) 和 `bits.c` 中各函数的题目要求。
- 课程讲义《信息的表示和处理(1)》《信息的表示和处理(2)》《信息的表示和处理(3)》（课堂提供）。